

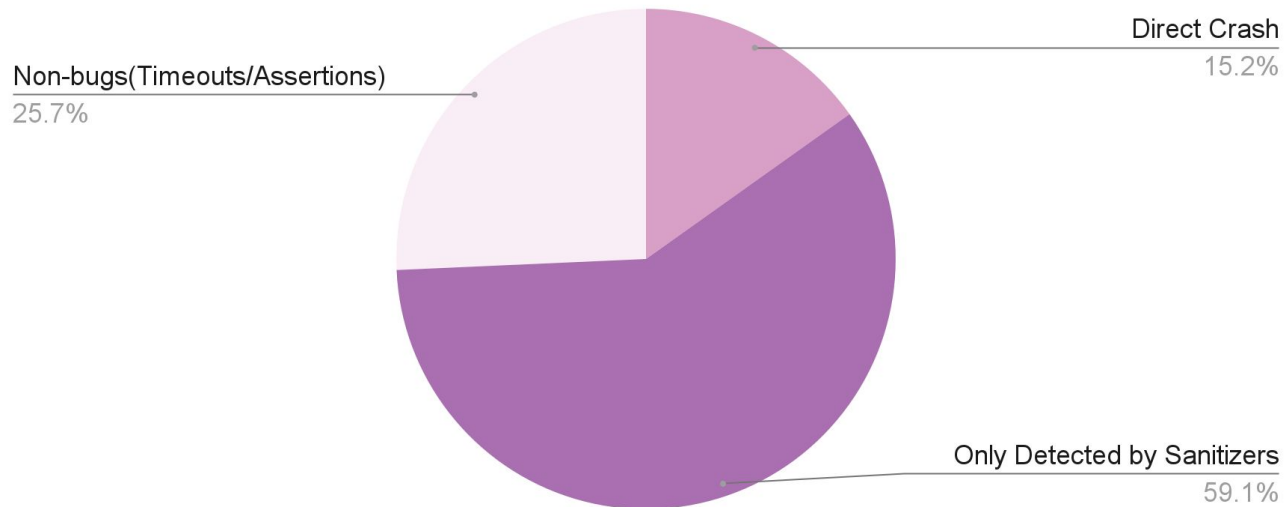
Sand: Decoupling Sanitization From Fuzzing For Low Overhead

Ziqiao Kong^{*}, Shaohua Li^{*}, Heqing Huang, Zhendong Su

ICSE'25

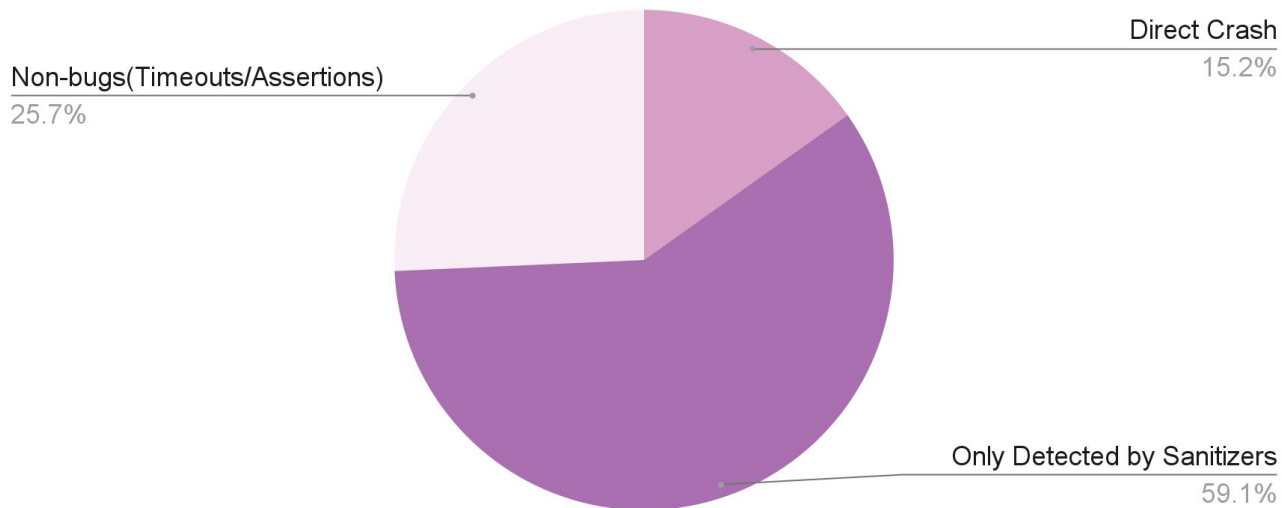
Motivation

Motivation: Sanitizers



The number of different types of bugs found by OSS-Fuzz

Motivation: Sanitizers



The number of different types of bugs found by OSS-Fuzz

Fuzzing without sanitizers = Miss many bugs



Motivation: Gaps

- 99% inputs do not trigger bugs
- If checking sanitizers for every input:
 - Address Sanitizer - 3.22x slowdown
 - Undefined Behavior Sanitizer - 1.92x slowdown
 - Memory Sanitizer - 45.52x slowdown



Motivation: Gaps

- 99% inputs do not trigger bugs
- If checking sanitizers for every input:
 - Address Sanitizer - 3.22x slowdown
 - Undefined Behavior Sanitizer - 1.92x slowdown
 - Memory Sanitizer - 45.52x slowdown



Motivation: Gaps

- 99% inputs do not trigger bugs
- If checking sanitizers for every input:
 - Address Sanitizer - 3.22x slowdown
 - Undefined Behavior Sanitizer - 1.92x slowdown
 - Memory Sanitizer - 45.52x slowdown

Insight: Not every input worth sanitizing



Motivation: Gaps

- 99% inputs do not trigger bugs
- If checking sanitizers for every input:
 - Address Sanitizer - 3.22x slowdown
 - Undefined Behavior Sanitizer - 1.92x slowdown
 - Memory Sanitizer - 45.52x slowdown

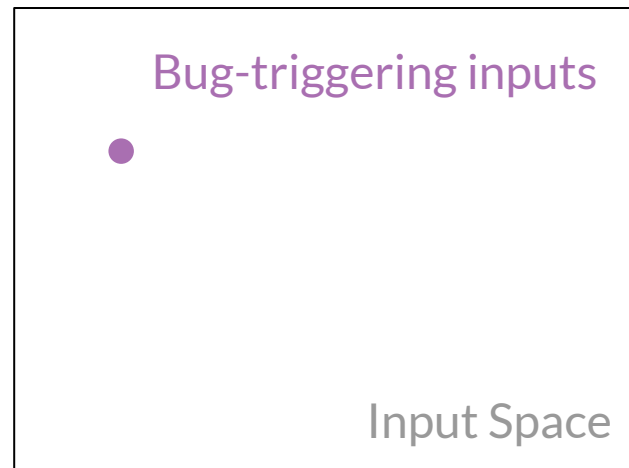
Question: How to filter out inputs worth sanitizing?

Approach



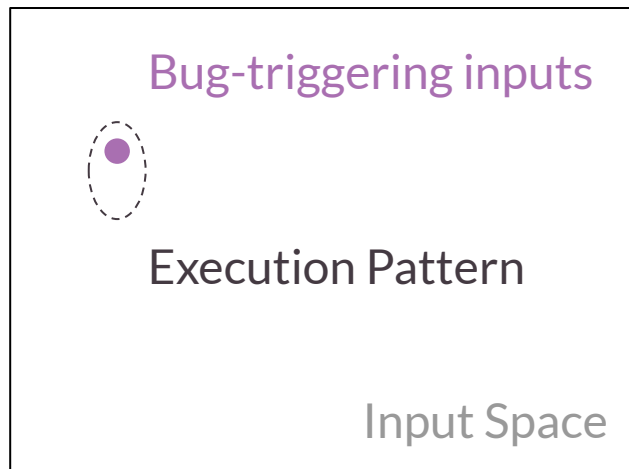
Approach: Execution Pattern

- Only sanitize executions triggering bugs?
 - Obvious paradox



Approach: Execution Pattern

- Only sanitize executions triggering bugs?
 - Obvious paradox
- Our approach: Execution Pattern
 - Captures all bug-triggering inputs
 - “Predict” if an input could trigger a bug
 - *Without execution*
 - Only sanitize partial inputs





Approach: Execution Pattern

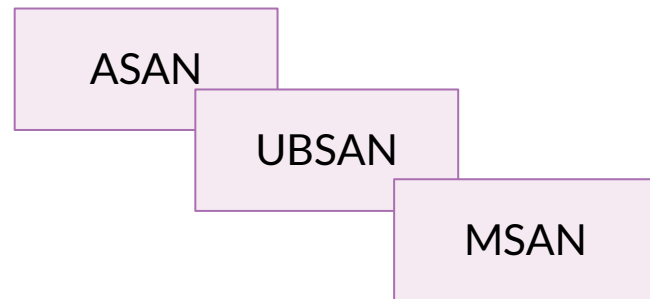
- Execution Pattern
 - Technically: Simplified map in AFL
 - Algorithm: `hash(map)` where `map[i] = (map[i] == 0 ? 0x1 : 0x80)`
- Only inputs with unseen execution pattern are sanitized
 - Different basic block sequences of a trace
 - Superset of corpus items (coverage increasing)



Approach: Execution Pattern



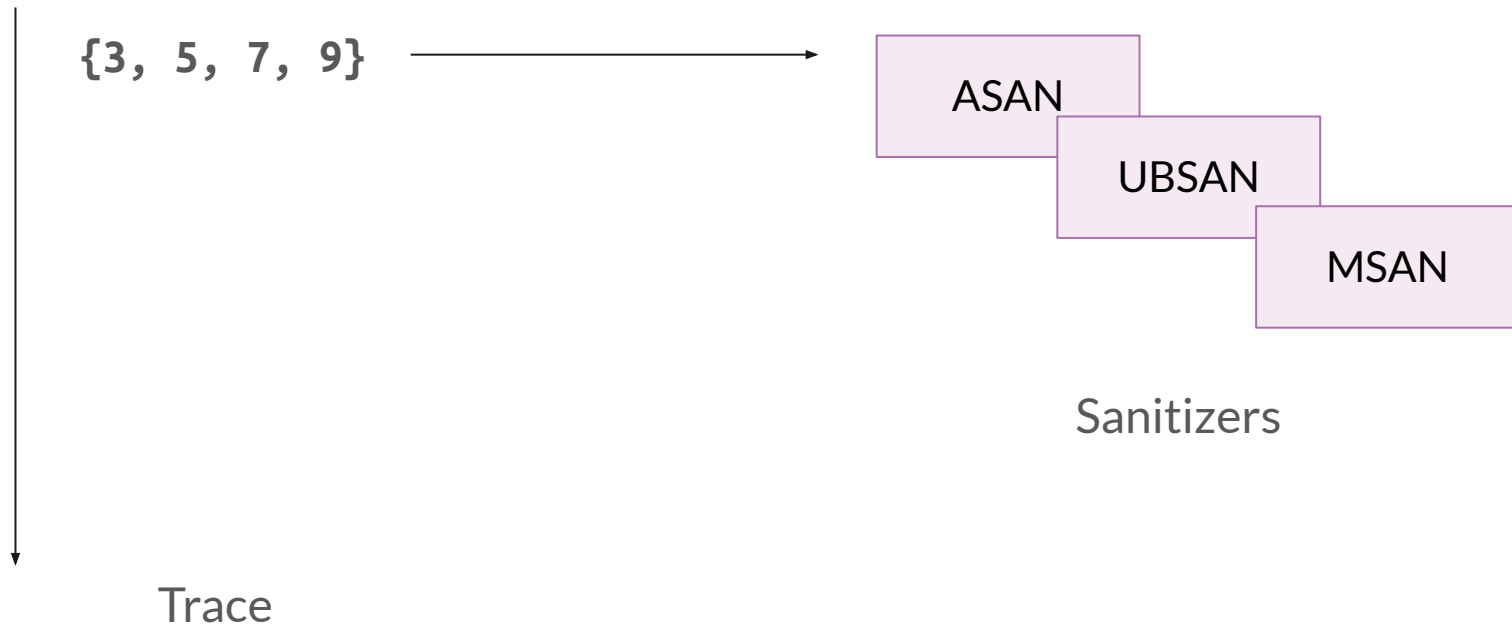
Trace



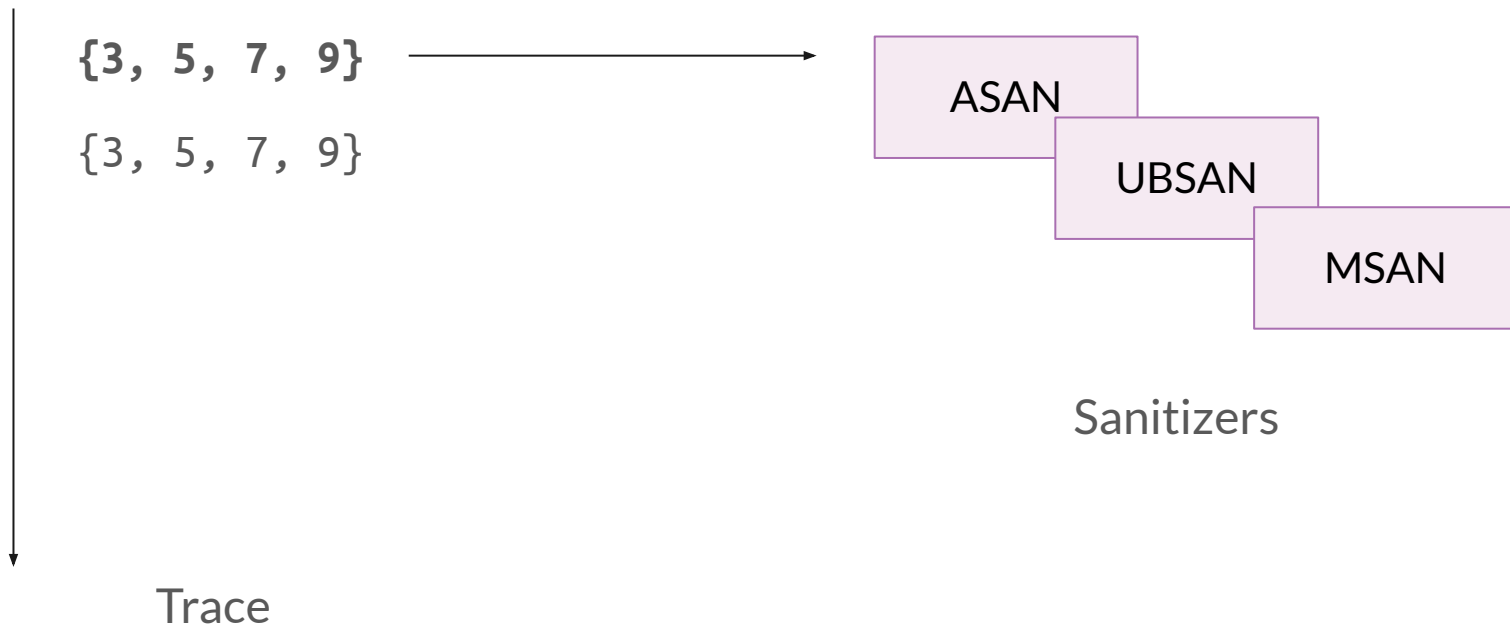
Sanitizers



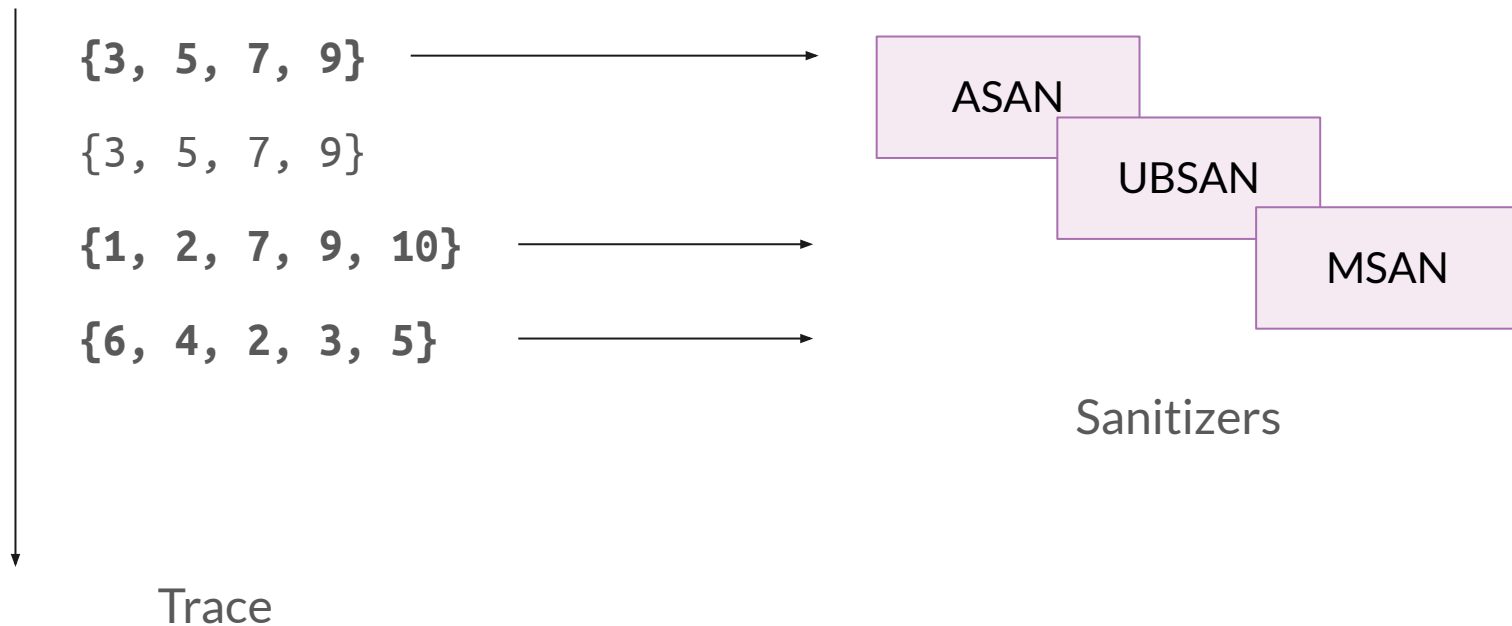
Approach: Execution Pattern



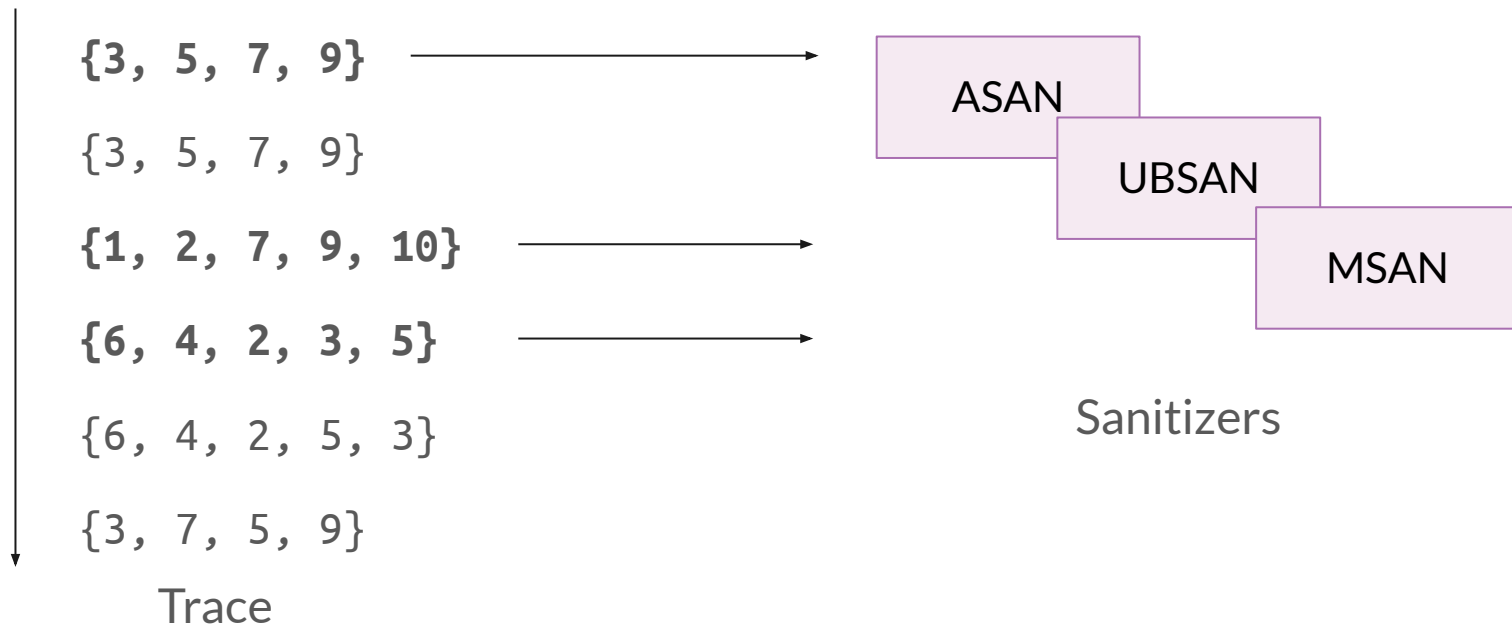
Approach: Execution Pattern



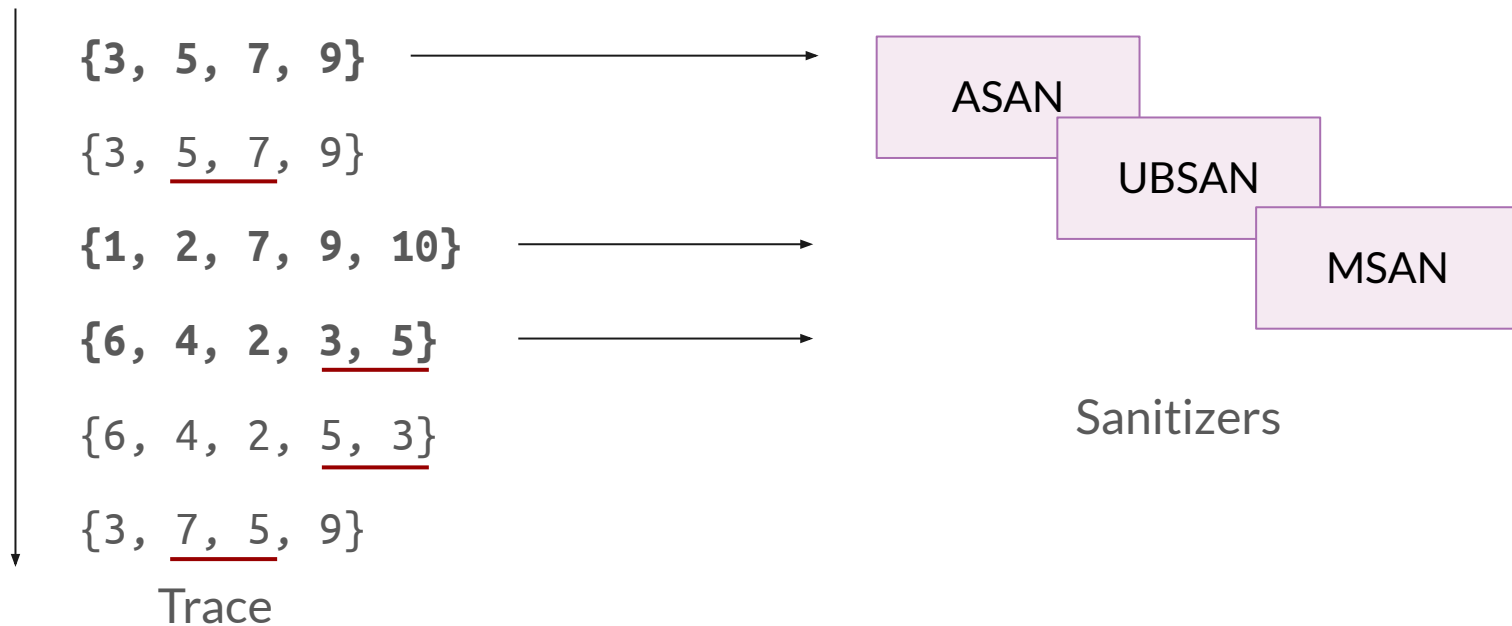
Approach: Execution Pattern



Approach: Execution Pattern

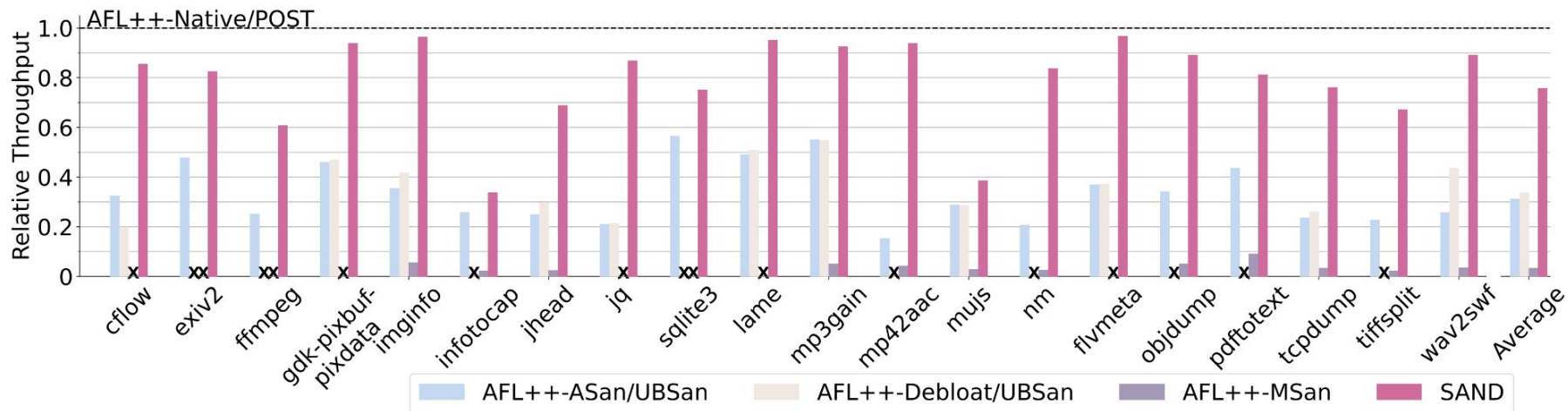


Approach: Execution Pattern

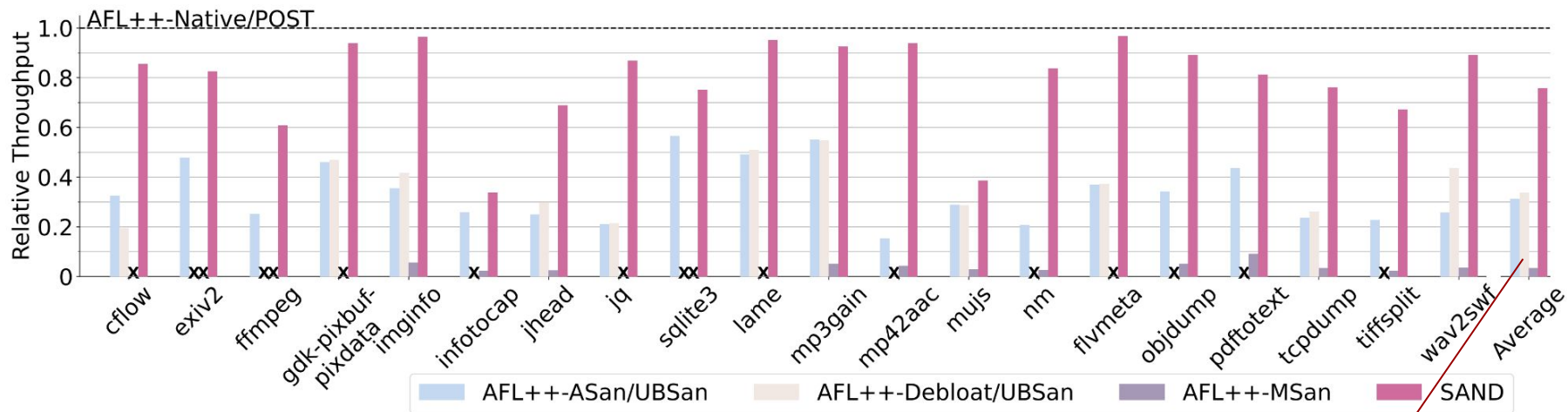


Evaluation on Unibench

Throughput



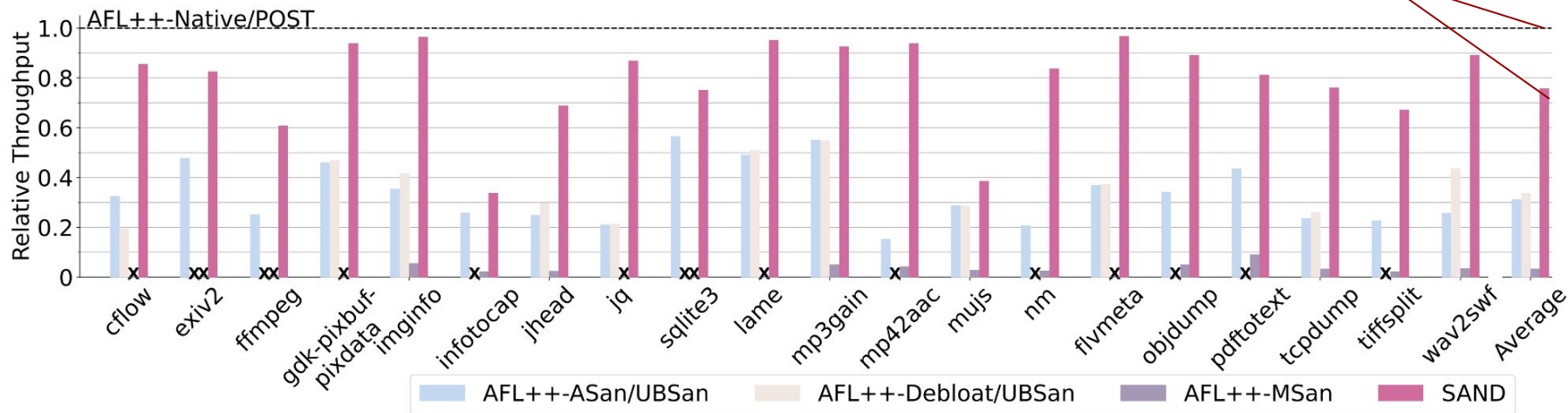
Throughput



2.4x, 20x speedup compared to ASAN and MSAN

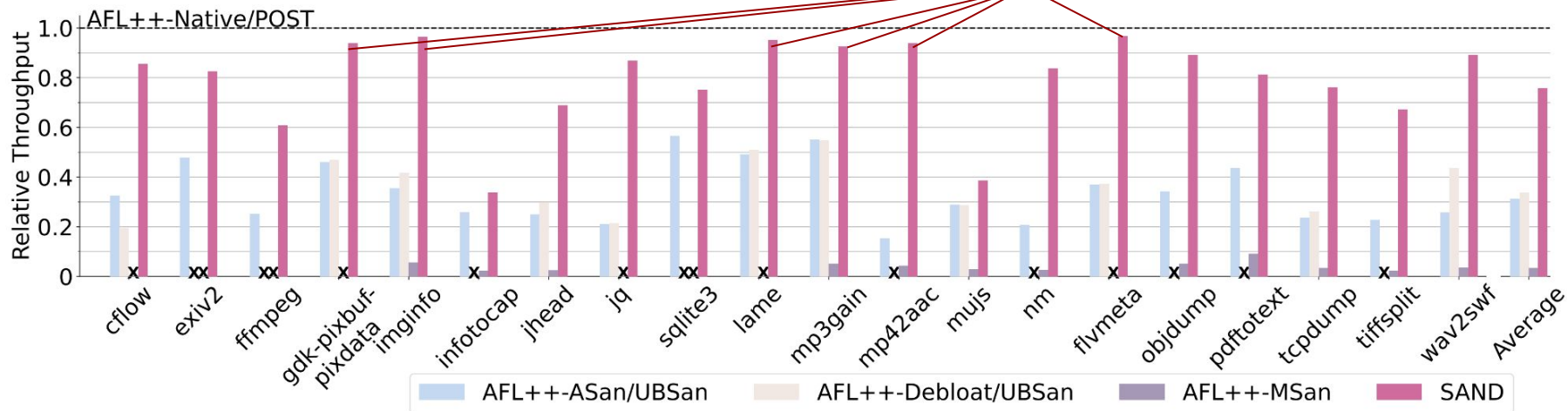
Throughput

75% throughput compared to AFL++ w/o sanitizers



Throughput

Almost same throughput on best cases





Bugs & Coverage

- Captures **all bugs** of ASAN/UBSAN/MSAN/Debloat
- Additional 30 bugs compared to all baselines
- Find the highest number of bugs on 18 targets out of 20 targets
- Reveal 9 new bugs on these heavily targets
- Best coverage over all baselines (except native) on all 20 targets
 - Thanks to the high throughput

Discussion



Alternative Pattern

- Execution pattern only captures 92% bug-triggering inputs
- Coverage increasing
 - Evaluated and miss 15% bugs
- Classified coverage map
 - Capture 99.9% bug-triggering inputs, superset of execution pattern
- Other more tight and precise pattern?



Applications

- SAND allows freely combine testing oracles
 - ASAN and MSAN, which are not compatible during compilation
- Other sanitizers including previous advancements
- Other fuzzers/schedule
 - MOpt
 - Fuzzilli

Artifacts

- Upstreamed to AFL++ (4.31c) and LibAFL (0.15.2)
- Available, Reusable, Functional
- Fully open-source!
- Link: <https://github.com/wtdcode/sand-aflpp>



Q&A

Appendix



Coverage Increasing?

- Execution pattern is super set of coverage increasing
 - {1, 2, 3} - coverage increasing
 - {1, 2, 3, 4} - coverage increasing
 - {3, 4} - **not coverage increasing but execution pattern**



Soundness

- Is execution pattern sound? Does it miss bugs?



Soundness

- Is execution pattern sound? Does it miss bugs?
- *Unfortunately, no theoretical guarantee*



Soundness

- Is execution pattern sound? Does it miss bugs?
- *Unfortunately, no theoretical guarantee*
 - Our evaluation proves it does not miss
 - SAND might miss crashes, but *capturing one crash is enough to capture the bug*
 - A more loose pattern might be explored



Collision

- Is there collision existing for execution pattern?
 - Collision: A & B having different execution pattern but same hash
 - In theory yes, *but in practice no*
 - Nearly no hash collisions (only 4 of all targets) found



Targets that SAND behaves bad

- Ratio of inputs with execution pattern is high
 - High complexity, like mujs and ffmpeg
- MSAN is way too slow in most cases
 - Without MSAN, throughput would be much higher